Omelets Need Onions

E-graphs Modulo Theories via Bottom Up E-Matching

Philip Zucker
philzuckerblog@gmail.com

Abstract

E-graphs[18] are a data structure for generic equational reasoning and optimization over ground terms. One of the benefits of e-graph rewriting is that it can declaratively handle useful but difficult to orient identities like associativity and commutativity (AC) in a generic way. However, using these generic mechanisms is more computationally expensive than bespoke routines. There exist specialized efficient algorithms and data structures for terms containing sets, multisets, linear expressions, polynomials, and binders. A natural question arises: How can one combine the generic capabilities of e-graph rewriting with these specialized theories? This talk discusses a pragmatic approach to e-graphs modulo theories (EMT) using two key ideas: bottom up e-matching and semantic e-ids.

Keywords

e-graphs, e-matching, term rewriting, equality saturation, SMT, union find, completion, theory solvers, Gröbner basis

ACM Reference Format:

Philip Zucker. 2025. Omelets Need Onions E-graphs Modulo Theories via Bottom Up E-Matching. In . ACM, New York, NY, USA, 7 pages. https://doi.org/10.1145/nnnnnnn.nnnnnnn

0.1 Terms Modulo Theories

It is instructive to dial back from the complexity of e-graphs to see how one can make sense of the "baking in" of theories for ordinary tree-like terms.

The most basic form of a term is a function symbol and an order list of children

```
type term = Fn of string * term list
```

We can extend this to a more general form of term

```
type term =
  | Fn of string * term list
  | MS of string * term multiset
  | Set of string * term set
  | Poly of string * term poly
  | LitInt of int
```

The specialized data structures like set and multiset can be implemented using regular lists. Canonical list representatives can be found by using sorting and deduplication. A more performant thing to do can be to use Patricia tries, which are also structurally canonical but support faster set and map operations.

One can also use smart constructors [22] to apply built in rewrite rules immediately upon construction. They check all rules which have the smart constructor symbol at the head of the pattern. If the rewrite rules are convergent, one can maintain the invariant the terms are only stored in canonical form. This can be mixed and matched with the previous approach, as the container structures can be encoded into the terms.

If the terms are stored in a structurally canonical way, one can use regular hash consing [5] to intern them. This then support fast equality checks modulo the theories.

This shows that equality checking and interning term modulo theories is not hard, but what about pattern matching?

[13] [26]

0.2 What patterns are possible?

Unification and pattern matching are a logical and syntactic slant on the notion of equation solving **?? ??**. The problems are formulated as goal equations to be solved, and the solutions are substitutions, mappings from variables to terms. The operate as backwards reasoning method that breaks down goal equation sets until it is refined into a substitution.

E-matching in the e-graphs community may have a connotation of matching a pattern against an e-graph, but in the term rewriting community it has the connotation of solving a pattern matching problem modulo an equational theory E. The e-graph version of e-matching is related to e-matching a term modulo the theory of ground equations that the e-graph represents.

Equational theories fall into different classes of how many solutions they may allow for their E-unification or E-matching problems. Some theories will only have a finite set of solutions, some a countably infinite set, and others an uncountably infinite set.

First order syntactic unification is for equation solving for a very rigid, basically structural theory of terms. It is unusual in the spectrum of pattern matchers in that it either succeeds once or fails. For example, the goal $cons(X, Y) =^? cons(zero, nil)$ is solved by the substitution $\{X \mapsto zero, Y \mapsto nil\}$ and the goal $cons(X, X) =^? cons(zero, nil)$ has no solution.

Another case may be that a finite number of solutions must be returned. This is the case for some structural theories like AC symbols, sets and multisets. The goal $\{X, Y\} =^? \{1, 2\}$ is solved by $\{X \mapsto 1, Y \mapsto 2\}$ and $\{X \mapsto 2, Y \mapsto 1\}$.

For other theories, an enumerably infinite number of solutions is possible. $X + Y =^? 42$ considered over the integers has solutions $\{\{x \mapsto n, Y \mapsto 42 - n\} | n \in \mathbb{Z}\}$. $X \cap Y =^? \{1, 2, 3\}$ has solutions $\{X \mapsto A, Y \mapsto B | \{1, 2, 3\} \subseteq A, B\}$.

Finally, even this may not be possible for some theories. Consider solving $X + Y =^? \pi$ over the $\mathbb{R}$. The only remaining possibility is

Conference'17, Washington, DC, USA

ACM ISBN 978-x-xxxx-xxxx-x/YYYY/MM
https://doi.org/10.1145/nnnnnnn.nnnnnnn

to instead return a constraint, which more of less just returns the pattern.

In short, e-matching modulo theories on terms runs the spectrum from expensive to hopeless.

1 Bottom Up E-Matching Plays Nicer With Theories

One way to win is to refuse the play the game.

The statement of the pattern matching problem gave a set of equations to be solved or a pattern to be matched against a particular term.

However, in equality saturation one is matching against an entire term bank in the form of the e-graph. This problem can be reduced to the previous problem by a scan of all terms, but this is not necessarily the only method.

A different formulation of the pattern matching problem that solves the issues of top down matching is the bottom up matching procedure. Instead of being given a term and a pattern to match it to, one is instead given a term bank T with which to attempt to fill the pattern variables to find a match. [23]

For different patterns and termbanks, one method or the other may be more performant.

A bad pattern for top down e-matching would be a deep one with few variables. For example,

$$foo(foo(foo(foo(foo(foo(X))))))$$

.

A bad pattern for bottom up e-matching would be a shallow one with many variables. For example,

$$foo(bar(X), bar(Y), bar(Z), bar(W), bar(V))$$

As a rough model of the situation, consider an e-graph with E e-classes and N e-nodes. On average, each e-class has N/E e-nodes.

Top-down e-matching pays a factor E for a scan of all e-classes to start to pick a root e-class to pattern match over. Every time top-down e-matching expands an e-class into it's e-nodes, it adds a multiplicative factor due to this searching. Because of this, top-down e-matching is exponential in cost in terms of depth of the pattern d. The cost of top down e-matching is something like $O(E(\frac{N}{E})^d)$.

Bottom-up e-matching pays for a scan for the number of variables V in the pattern. The constructing the instantiated pattern is just a lookup and pays the cost of a hash table or indexing tree, which is typically something like $O(\ln(N))$. The cost is therefore something more akin to $O(E^V d \ln(N))$.

A reason that top down matching may feel natural is that makes sense for pattern matching against a pointer chasing implementation of single term. This is the typical situation in many programming language like contexts.

Another reason bottom up e-matching may not feel natural is that the most commonly considered termbank is one consisting of of ground syntactic terms with no theories. In this case $\frac{N}{E} = 1$ and the exponential character of top down matching is gone. It is a dominant method. People do not necessarily use simple syntactic terms because they do not want baked in theories. They do it because baking in theories is complex and expensive. Bottom-up e-matching ameliorates these issues.

Bottom up e-matching can be seen as a particular query plan for relational e-matching using a generic join [17]. Likewise Top down e-matching can be seen as the same.

The positive perspective of relation e-matching [19] is that it gives one freedom to find a good efficient query plan. A negative perspective of relational e-matching is that it doesn't commit to a choice, leaving behind a vague gesturing to the new problem of query optimization which is yet another subsystem to design and implement.

Bottom up-matching has the killer feature is that it proceeds in the same direction as adding terms into the egraph. Therefore it uses the same indexing structures and canonization that already exist. Bottom up e-matching grounds pattern variables as quickly as possible, and ground terms are the e-graph's bread and butter. Ground is good.

Top-down e-matching requires the extra ability to expand from e-classes to e-nodes and from e-nodes to their children. This is as we have seen, extra challenging or even impossible in the face of theories. Top-down keeps the pattern in an ungrounded state as long as possible. Unground is bad.

Bottom up e-matching can emulate the results of top-down e-matching. If it is possible to enumerate the subterms of your target term, you can fill a term bank with these subterms. Bottom up e-matching will then find a superset of the same matches top-down will. In the presence of some theories enumerating subterms is not possible or meaningful and top-down e-matching will not work at all. You can still bottom up match over a term databank. In this sense, bottom up e-matching is more powerful than top-down e-matching.

```
# A shallow bottom up embedding of the rewrite
# foo(bar(X), Y) -> biz(X)
for X in eclasses:
    for Y in eclasses:
        try:
            # possibly failing lookup in hashcons
            lhs = foo[bar[X], Y]
            # construct rhs
            rhs = biz(X)
            # set equal
            egraph.union(lhs,rhs)
        except e:
            pass
```

The maximally naive bottom up e-matching that scans over all e-classes can be improved by filtering on the intersection of all the tables where the variable occurs and by pushing intermediate constructions upwards in the loop ordering.

```
for X in set(bar.keys()).intersection(biz.keys()):
  try:
    t1 = bar[X]
    for Y in set([k[1] for k in foo.keys()]):
      try:
          lhs = foo[t1, Y]
          rhs = biz(X)
          egraph.union(lhs,rhs)
```

2 Generalized Union Finds and Semantic E-ids

E-graphs systems are the interplay of two entities, e-nodes and e-classes. A natural approach to generalize e-graphs is to consider generalizing one or both.

After bottom up e-matching, the second key ingredient for e-graphs modulo theories is to generalize the e-id and the union find. These become structured e-ids / semantic e-ids / values. The union find becomes a theory specific canonizer that supports equality assertions.

A first way to see this might make sense is to note that a simple e-graph can be seen as a uninterpreted minimal model $\mathcal{M}$ of partial functions. The collection of e-ids $[V]$ corresponds to the set of values in the model and the e-node tables correspond to the data of the model of the partial functions $[\![f]\!]$. One can ask for interpretation onto other domains other than an uninterpreted one where the values have no structure except equality. One can consider semantic domains like polynomials, linear expressions, terms, constructors, sets, multisets and so on. These corresponds to structured e-ids. Mutiple kinds of strucutured e-ids can be most simply supported by using multi-sorted logic. Expressions of different sorts are restricted by being equated by a static type check and the role of mediating between these sorts is perfomed by the e-nodes and congruence closure over e-nodes.

Another point is to note that even in hash consing for terms modulo theories, there is utility in having both interned and non interned forms of terms. A smart constructor should not bother interning the intermediate expressions until they have been canonized. The e-nodes are the interned forms and the structured e-ids are the non-interned forms. The structured e-ids are in a sense kept in extracted non-interned term-like form for easy deep inspection and theory specific destructive manipulation.

Another way to see that this makes sense is to ask "what interface does the union find present?". The internals of the union find and the structure of the e-ids is irrelevant to how they are used in the e-graph. At minimum they give an interface

```
type t
type eid
val create : unit -> t
val eq     : eid -> eid -> bool
val fresh  : t -> eid (* makeset *)
val canon  : t -> eid -> eid (* find *)
val assert_eq : t -> eid -> eid -> unit (* union *)
```

This API is largely the same as that required by the theory of an SMT solver minus the support for backtracking. It is particularly close to the interface of Shostak style theory combination. This supports the the schematic equation $EMT = SMT - SAT$.

2.1 Union Finds solve Ground Atomic Equations

The union find can be seen as a normal form or canonizer of a system of atomic equations. The knuth bendix completion of a ground atomic equational theory is guaranteed to terminate and the resulting rewrite system can be interpreted as a union find [25].

Consider for example the atomic ground equation system

$$\begin{aligned} e_1 &= e_2 \\ e_2 &= e_3 \\ e_2 &= e_1 \\ e_3 &= e_1 \\ e_4 &= e_5 \end{aligned} \tag{1}$$

Running Knuth Bendix completion with an atomic term ordering

$$e_i < e_j \triangleq i < j$$

results in an atomic ground rewrite system

$$\begin{aligned} e_2 &\longrightarrow e_1 \\ e_3 &\longrightarrow e_1 \\ e_5 &\longrightarrow e_4 \end{aligned} \tag{2}$$

The $\longrightarrow$ can be represented by a parent pointer in a union find.

However, atomic ground theories are not the only ones that are guaranteed to be completable to canonizers. There is a spectrum of theory specific "union finds" that support canonization and ground equality assertion [12]. Many of them have the flavor of a specialized theory specific ground Knuth Bendix completion.

2.2 Example Theories

- $\mathcal{V} = \text{span}_{\mathbb{Q}}\{e_1, e_2, e_3, ...\} \Big/ V_E$ Linear equations can be put into row echelon form. The row echelon form can be used to canonically quotient a vector space by a subspace. a set of linear equations can be put into the matrix form $Ax = 0$. Each equation of the above example system can be encoded as a row of a matrix.

$$\begin{bmatrix} 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 \\ -1 & 1 & 0 & 0 & 0 \\ -1 & 0 & 1 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 \end{bmatrix} \begin{bmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \end{bmatrix} = 0$$

  The row echelon form corresponds to a ground rewrite system. It can be used starting from the bottom to canonically normalize $\sum_i a_i e_i$ expressions where $a_i \in \mathbb{Q}$. For example $e_1+3e_3+5e_5 \longrightarrow e_1+e_3+5e_4 \longrightarrow e_1+3e_2+5e_4 \longrightarrow 4e_1+5e_4$. The method however works just as well with asserted equations like $4e_1 + 5e_4 = 13e_2 + 2e_3$, extending the capabilities of the union find to bake in linear expressions.

$$\begin{bmatrix} 1 & -1 & 0 & 0 & 0 \\ 0 & 1 & -1 & 0 & 0 \\ 0 & 0 & 0 & 1 & -1 \end{bmatrix} \begin{bmatrix} e_1 \\ e_2 \\ e_3 \\ e_4 \\ e_5 \end{bmatrix} = 0$$

- $\mathcal{V} = \mathbb{Q}[e_1, e_2, ..] \Big/ I_E$ Gröbner bases are the generalization of row echelon form the multivariate polynomial equation systems. This is particularly useful because much of trigonometry and Euclidean geometry can be put into a form that is essentially polynomials for example $\sin(x)**2+\cos(x)**2 = 1$.

- Ground multiset equations are completeable $\{\}a, b, b\}\} = \{c, d\}$. The structured e-ids are multisets. This gives each sort one special associative and commutative operation. [27] This can be seen as a special case of Groöbner basis calculation as each monomial $x^n y^m z^k$ can be seen as a multiset of it's variables with the multiplicity given by the exponent. Indeed, the termination of Buchberger's algorithm is guaranteed by multiset orderings on the produced polynomials. It can also be seen as integer lattice rewriting via Hilbert or Graver bases and has connections to the theory of integer programming [15]
- Ground terms equations. In an amusing self reflection, the theory solver could itself be an e-graph, which is a canonizer for ground term equations. It is unclear why this would be useful. One could separate functions symbols to those considered outer and inner. This would enable for example, two e-graph implementations to interact with each other.
- Atomic Completion modulo a group action. A union find can be built that carries group elements on it's edges [6] [11] [20]. The composition law of the group enables composition while chasing up the parent pointer of the union find, and group inversion enables turning comparison to the root back into comparison with a leaf of the union find. In this manner it can be used to store an equivalence relation modulo a group action. For example the group of integer addition 'x + 6 = y - 3' while subsumed by the Row echelon method can be more simply and efficiently implemented using this technique. Permutation groups are also very useful in the context of nominal rewriting and slotted egraphs.
- Atomic completion modulo constructors and primitives. This is a subcase of ground completion where only atomic entities are unioned and with built in injectivity rules for constructors. The semantic domain is $\mathcal{V} = \{e_i\} \uplus \mathcal{P}$. E-class ids may either be uninterpreted or grounded to a concrete entity. Tie breaking of the union operation on the union find has to be adjusted such that the ground entity is always the root. Unioning two compatible constructors can be resolved by syntactic unification. Unioning two incompatible constructors is considered an error. Constructors and injectivity can be encoded into the e-graph and rules, but this is a form of baking them in. Having the constructor term reified makes occurs checks and bisimilarity compaction for coalgebraic datatypes easier to perform by ordinary term routines. This is related to the ideas of Co-egraphs [24] which add a cofunction/observation table in addition to the enode function table. The identification of e-ids with values suggests a method to support lambda terms by using Closure values as structured e-ids in an intriguing connection with the ethos of Normalization by Evaluation. In NbE, the semantic domain is extended by syntactic forms representing stuck terms.
- A Boolean theory of ground clauses $\neg a \vee b \vee c$ or ground Horn clauses $b \wedge c \implies a$ can be canonized by ordered resolution. This can also be extended to ground superposition clauses $\neg(x = y) \vee z = x \vee c = true$. This may give a method to encoding contextual equality.
- Disjoint signature of terms with a convergent rewrite system.

It is possible to use theory solvers that may not terminate. One may either hope they do, or terminate their completion process early and accept the incomplete canonizers. The e-graph typically has a self-healing character in the face of incomplete canonization.

- String Rewriting $< e_1, e_2, ... | R_E >$. The completion of string rewriting in general will not complete because string rewriting is rich enough to encode turning complete computation, and successful completion would give a decision procedure for . This is an important counterexample as string rewriting can be viewed as ground equations + associativity. Because of this, a fully satisfying solution to an e-graph (which represent ground equations) with baked in associativity is unlikely to be possible. Sequence e-graphs. This may be useful for modelling programs which often need sequencing of side effectful instructions like in Denali. Two related theories are non-commutative rings and finitely presented groups, both of have a flavor of sequences and are not decidable.
- The author is uncertain if it is in the literature if the word problem of differential rings is undecidable, but it probably is. This suggests the possibility however of a method to bake in some notions of calculus and differentiation into the e-graph.
- Knuth Bendix Completion - One may choose e-ids to be so structured as to be terms themselves. Unorientable derived equations can be lifted to be equality saturation rules. This is related to ordered completion/rewriting. The ideas around knuth bendix completion are how destructive rewriting (demodulation) can be used while retaining completeness in equtional theorem proving systems. A given set of desirable destructive rewrite rules may be completed ahead of time, but adding in new ground equations in general. The term ordering used to orient and complete the original system can also be determined ahead of time. Strongly term decreasing simplification rules may prefer suing Knuth Bendix orderings which orient roughly speaking by term size. Knuth Bendix completion is such a powerful method that it isn't clear there is an advantage to having the outer e-graph layer.

An e-graph rewriting system may want to mix and match these possibilities. The most straightforward design is to associate a theory with each sort. The sort object is responsible for canonization and maintaining to normalizer data structures. It is not always the case that it makes sense to have one privileged notion of AC for example per sort.

3 Alternative Approaches

3.1 Extract, Simplify, and Insert

While the e-class indirection makes theory normalization inside the e-graph confusing, it is known how one would normalize an AST-like term. Associativity and commutativity can be dealt with by flattening and sorting an AST into a vector and there are plethora of techniques for beta normalization of terms.

A simple idea is to replace equality saturation rules encoding this behavior with an extraction, simplification, and reinsertion of terms. This allows many directed normalizing steps to be taken at once without bloating the e-graph with administrative junk.

A downside of this technique is that the intermediate states may not really be junk. For AC, the intermediate states may bubble together or collect up terms in such a way as to unlock new optimizations.

Koehler et al describe an "extraction-based substitution" [9] method for dealing with substitution of lambda terms. This method works for all rewrite theories for which one wants to avoid junk, not just lambda terms.

3.2 Constraints via sidecar SMT

A prominent useful class of automated reasoning solvers are Satisfiability Modulo Theories (SMT) solvers [2]. A SAT solver and an e-graph are communication fabric by which theory specific solvers talk to each other. The SAT solver leads to SMT having a backtracking flavor whereas e-graph rewriting has a saturation flavor. A reasonable approach to E-graphs modulo theories can be found in the motivating schematic equation $EMT = SMT - SAT$. If one only asserts unit clauses to an SMT solver then the SAT solver is not needed.

The core technology of SMT solvers is largely the same, but optimized for different use patterns and the first order model semantics is not the desired one for the purposes of optimization.

Upon satisfiability, SMT returns an arbitrary model. Equality saturation specifically wants a justified or minimal model, one in which every equality is forced by the constraints. As an example, given the problem ‘a = b‘ ‘c = d‘ an SMT solver may return a universe containing only a single element ‘a : v!0, b : v!0, c: v!0, d ; d!0, identifying everything with everything.

The e-matching d the e-matching algorithms are syntactic and furthermore CVC5 and Z3 do not necessarily make theory specific operators like integer addition available to the e-matcher. Alt-ergo is an exception.

Nevertheless, a technique for EMT is to maintain an equality saturating e-graph and to mirror all the equalities and theory specific facts into a side car SMT solver. The utility of this is to enable SMT queries, specifically in guards of equality saturation rules. Guards are distinguished from multipatterns in that no new pattern variables may be bound in them.

SMT solvers supply a black box for checking equality of two terms modulo the asserted axioms and theories. If one asserts $t \neq s$ and receives an unsat result, then the axioms justify $t = s$ in the minimal model. Given a term bank and bottom-up e-matching, this is sufficient to pattern match, albeit while requiring at worst case T SMT queries. With the addition of using unsat cores, this can be made more query efficient by querying many equalities at once.

??

```
(push)
(assert (not guard))
(check-sat)
(pop)
```

This technique has analogs in the form of Constraint Logic Programming [8] or Constrained Horn Clauses, or constrained rewriting [10].

4 Containers and Generalized ENodes

An alternative to consider from structured e-ids is generalized e-nodes. The basic idea is to merge the concepts of e-node and container primitive.

A standard e-node is a function symbol and a fixed number of ordered children.

Similar to the datatype for terms modulo theories, it is natural to consider an e-node that holds it's children in other data structures like sets, multisets, vectors or polynomials. This is a every so slightly shifted design from putting those structures in the e-ids. During rebuild, the children containers of e-nodes need to be rebuilt. Bottom-up e-matching still works and for some containers top-down e-matching can still work if the interface for enumerating children is extended to be able to return a list.

5 Just Use Terms Modulo Theories

6 Related Work

Other examples of e-graph extensions that are evocative of the structured e-id idea are colored e-graphs [14] which tag e-ids with a notion of context and slotted e-graphs which have some of the flavor of both generalized e-ids and generalized e-nodes.

There have been a number of extensions to rewriting to term rewriting modulo theories. Chapter 11 of Baader and Nipkow has a survey of some options [1]. Of particular interest is Normalizing rewriting [12] which has the most similarity to the approach here.

String Knuth Bendix systems appear in computational group theory applications. These systems by construction have built in associativity axioms.

Vampire and the E theorem prover are two powerful automated theorem provers with some built in support for AC symbols.

There is a line of work for congruence modulo AC applied in the alt-ergo SMT solver [3]

The extreme sharing of the e-graph is both its benefit and its problem. There are other term rewriting methods that can handle difficult to orient rules or baked in ness without going

Relational AC matching [21] uses the ideas of Relational E-Matching [19] to encode AC matching into database queries.

7 Conclusion

Bottoms up! Go forth and saturate!

Acknowledgments

I'd especially like to thank Cody Roux for many many belligerant grillings about term rewriting. I'd also like to thank Max Willsey, Yihong Zhang, Yisu Remy Wang, Graham Leach-Krouse, Greg Sullivan, Caleb Heibling, Sam Lasser, and Ryan Wisnesky.

References

[1] Franz Baader and Tobias Nipkow. 1998. *Term rewriting and all that.* Cambridge University Press, USA.

[2] Clark Barrett and Cesare Tinelli. 2018. *Satisfiability Modulo Theories.* Springer International Publishing, Cham, 305–343. doi:10.1007/978-3-319-10575-8_11

[3] Sylvain Conchon, Évelyne Contejean, and Mohamed Iguernelala. 2012. Canonized Rewriting and Ground AC Completion Modulo Shostak Theories : Design and Implementation. *Logical Methods in Computer Science* 8, 3 (Sept. 2012), 1–29. doi:10.2168/LMCS-8(3:16)2012 Selected Papers of the Conference *Tools and Algorithms for the Construction and Analysis of Systems* (TACAS 2011), Saarbrücken, Germany, 2011.

[4] David A. Cox, John Little, and Donal O'Shea. 2015. *Gröbner Bases*. Springer International Publishing, Cham, 49–119. doi:10.1007/978-3-319-16721-3_2
[5] Jean-Christophe Filliâtre and Sylvain Conchon. 2006. Type-safe modular hash-consing. In *Proceedings of the 2006 Workshop on ML* (Portland, Oregon, USA) *(ML '06)*. Association for Computing Machinery, New York, NY, USA, 12–19. doi:10.1145/1159876.1159880
[6] Thom Frühwirth. 2009. *Union-find algorithm*. Cambridge University Press, 256–280.
[7] Yeting Ge and Leonardo de Moura. 2009. Complete Instantiation for Quantified Formulas in Satisfiabiliby Modulo Theories. In *Computer Aided Verification*, Ahmed Bouajjani and Oded Maler (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 306–320.
[8] Joxan Jaffar and Michael J. Maher. 1994. Constraint logic programming: a survey. *The Journal of Logic Programming* 19-20 (1994), 503–581. doi:10.1016/0743-1066(94)90033-7 Special Issue: Ten Years of Logic Programming.
[9] Thomas Koehler, Phil Trinder, and Michel Steuwer. 2022. Sketch-Guided Equality Saturation: Scaling Equality Saturation to Complex Optimizations of Functional Programs. arXiv:2111.13040 [cs.PL] https://arxiv.org/abs/2111.13040
[10] Cynthia Kop and Naoki Nishida. 2013. Term Rewriting with Logical Constraints. In *Frontiers of Combining Systems*, Pascal Fontaine, Christophe Ringeissen, and Renate A. Schmidt (Eds.). Springer Berlin Heidelberg, Berlin, Heidelberg, 343–358.
[11] Matthieu Lemerre and Dorian Lesbre. 2024. Labeled Union-Find for Constraint Factorization. In *Proceedings of the 10th ACM SIGPLAN International Workshop on Numerical and Symbolic Abstract Domains (NSAD 2024)*. Pasadena, United States. HAL Id: cea-04996700.
[12] CLAUDE MARCHÉ. 1996. Normalized Rewriting: an Alternative to Rewriting modulo a Set of Equations. *Journal of Symbolic Computation* 21, 3 (1996), 253–288. doi:10.1006/jsco.1996.0011
[13] Krzysztof Maziarz, Tom Ellis, Alan Lawrence, Andrew Fitzgibbon, and Simon Peyton Jones. 2021. Hashing Modulo Alpha-Equivalence. arXiv:2105.02856 [cs.PL] https://arxiv.org/abs/2105.02856
[14] Eytan Singher and Shachar Itzhaky. 2023. Colored E-Graph: Equality Reasoning with Conditions. arXiv:2305.19203 [cs.PL]
[15] B. Sturmfels. 1996. *Grobner Bases and Convex Polytopes*. American Mathematical Society. https://books.google.com/books?id=K-bxBwAAQBAJ
[16] Remy Wang. 2025. Completeness of Equational Proofs with Equality Saturation. https://remy.wang/blog/birkhoff.html
[17] Yisu Remy Wang, Max Willsey, and Dan Suciu. 2023. Free Join: Unifying Worst-Case Optimal and Traditional Joins. arXiv:2301.10841 [cs.DB] https://arxiv.org/abs/2301.10841
[18] Max Willsey, Chandrakana Nandi, Yisu Remy Wang, Oliver Flatt, Zachary Tatlock, and Pavel Panchekha. 2021. egg: Fast and extensible equality saturation. *Proc. ACM Program. Lang.* 5, POPL, Article 23 (jan 2021), 29 pages. doi:10.1145/3434304
[19] Yihong Zhang, Yisu Remy Wang, Max Willsey, and Zachary Tatlock. 2022. Relational E-Matching. arXiv:2108.02290 [cs.DB] https://arxiv.org/abs/2108.02290
[20] Philip Zucker. 2022. Groupoid Annotated Union Finds. https://www.philipzucker.com/union-find-groupoid/
[21] Philip Zucker. 2023. Relational AC Matching. https://www.philipzucker.com/relational-ac-matching/
[22] Philip Zucker. 2024. Acyclic Egraphs and Smart Constructors. https://www.philipzucker.com/smart_constructor_aegraph/
[23] Philip Zucker. 2024. Bottom Up Egraph Ematching Plays Nicer with Theories (AC, etc). https://www.philipzucker.com/bottom_up/
[24] Philip Zucker. 2024. Co-Egraphs: Streams, Unification, PEGs, Rational Lambdas. https://www.philipzucker.com/coegraph/
[25] Philip Zucker. 2024. EGraphs as Ground Completion Talk and EGRAPHS 2024 notes. https://www.philipzucker.com/egraph2024_talk_done/
[26] Philip Zucker. 2024. Hashing Modulo Theories. https://www.philipzucker.com/hashing-modulo/
[27] Philip Zucker. 2024. Towards an AC Egraph: Groebner, Graver and Ground Multiset Rewriting. https://www.philipzucker.com/multiset_rw/

A Appendix 1

A.1 A No Go Counterexample

String rewriting is finding pattern strings and replacing them. It is fairly straightforward to encode a turing machine tape and head into a string [1]. The transitions of the head can be encoded as string rewrite rules. Because of this, string rewriting is Turing complete and we can't expect all equational theories generated by string equations to be decidable.

String rewriting can be shallowly embedded into term rewriting in multiple ways. One method uses a binary concatenation operation that is associative $(X \cdot Y) \cdot Z = X \cdot (Y \cdot Z)$. The remaining equations of a particular string rewrite system can be represented as ground term equations. If we take the view that an E-graph represents decision procedure for ground term equations then even before discussions of e-matching, an e-graph with a baked in notion of associativity cannot exist.

No No-Go theorem is ever actually a No-Go. It just becomes clear that you need to side step the preconditions or framing of the theorem to achieve what you want. Undecidability of the halting problem does disprove the existence of program analysis and the failure of single layer neural networks to encode xor does not say much about the ability of more complex ones.

It is known that [12] that ground completion modulo theories is undecidable for

- A
- Groups
- ACD

A.2 A Go Go Example

The meaning of the notation of conventional mathematics is often both contextual and ambiguous. Like English, this is not necessarily an impediment to effective communication between those with trained eyes. The polynomial expression $x^2 + 2x + 1 = 0$ may be making a statement about a particular point x, or about a surface, or describing an algebraic fact about elements of any element of some ring x. The former is more analogous to a ground equation in therm term rewriting sense and the latter is more like a non-ground universally quantified equation.

A system of polynomial equations can be converted into one that is equivalent but normalizing under the operation of multivariate polynomial division. This sets of polynomials is called a Gröbner basis [4]. The algorithm to do so is Buchberger's Algorithm. A Gröbner basis is the direct analog of a normalizing rewrite system and Buchberger's algorithm is the direct analog of Knuth-Bendix completion. Like ground Knuth Bendix completion, Buchberger's algorithm is guaranteed to terminate. Unlike ground knuth bendix completion, the complexity is unlikely to be polynomial.

While it is annoying to make the connection truly rigorous, it is clear that Buchberger's algorithm is performing ground completion modulo a baked in theory of rings.

Two special subcases of Buchberger's algorithm occur for linear polynomials and for polynomials consisting of only two monomials. The first is a perspective on Gaussian elimination. The latter can be seen as ground multiset rewriting or as integer lattice rewriting. It has connections to the theory of integer programming [15]

It is known that ground completion modulo theories is decidable for the theories of [12]

- AC
- ACU
- ACI
- ACN
- ACUI
- ACUN
- Boolean Rings

- Commutative Rings
- Finite Fields
- Abelian Groups

A.3 Completeness: Should One Care?

Naive equality saturation is not a complete equational theorem proving method because it is defeated by equations that have different variables on each side. [16]

Consider the theory $\forall X, foo(X) = a \; \forall Y, foo(Y) = b$. and the goal $a =^? b$. In naive equality saturation, these two equations can only be used as rewrite rules from left to right. Inserting the goal terms a and b into the e-graph's termbank will not be succiient to trigger the rules.

Either using unification in the form of superposition of unordered ocmpletion, or hvaing an enumeration method of ground terms is needed. [7]

The term banks of the EMT methods do not reduce the total number of e-classes if one wants to find the same number of solutions as naive equality saturation. The number of enodes is reduced.

In AC, if one genuinely wants to be able to search over all subsets of a multiset,

It is the author's belief that with a suffiicient term enumeratiom mechanism, the EMT methods above are also complete. Even the undecidable theories remain goal-complete if one allows for fair scheduling. These are complex claims that it is outside the scope of this paper to actually prove.

The main interest in understand the completeness of an EMT method relative to the naive algorithm not because the naive algorithm has good theoretical properties. it is that naive equality saturation is simple to explain, pragmatic, and efficient. Alternate EMT methods may have greater or less reasoning power than the naive algorithm on different problems. If the method is equally simple and more efficient that is fine.

Completeness is useful if you want absolute guanrantees you have the minimum possible term or that if the saturation saturates there is no solution.

Soundness however is table stakes.